Introduction solutions

Each identifier corresponds to the separate exercise file. Conditions and counterexamples are part of the answer.

OS01-E01 Enforcing a shared-machine boundary

1. Bob must not permanently monopolize the CPU/change privilege, arbitrarily read/write Alice's RAM, or arbitrarily access her files. These related boundaries have separate enforcement points.
2. Protected timer/mode and the kernel scheduler enforce CPU control. MMU/PTE permissions and OS-managed mappings enforce RAM access. Protected entry, trusted credentials, and ACLs/capabilities enforce file operations. Hardware does not directly interpret pathnames; it protects the checking kernel and metadata.
3. A globally shared clipboard or file improves convenience but adds authority and disclosure routes. Grant explicit rights to specified objects instead. Hiding GUI controls cannot enforce an API.

The answer must identify an enforcement point for each resource.

OS01-E02 Events, modes, and context

1. Timer interrupts are external/asynchronous; a system call is synchronous with the current instruction. DMA completion and CPU discovery are distinct events. Polling or interrupts can notify the CPU.
2. A timer interrupt in kernel mode requires neither user→kernel transition nor process switch. A cache-hit read normally has user→kernel→user, two mode transitions, and can return to the same thread with zero process switches. Additional scheduling is counted separately.
3. Polling completion status works if cache/memory ordering and buffer lifetime are correct. Poll frequency, event rate, detection latency, and CPU cost determine the trade-off. DMA does not require one interrupt per block.

Event classification does not substitute for counting actual transitions.

Introduction solutions (continued)

OS01-E03 Hash counterexamples and lifetime

1. Bucket1 is[5,9,13]. Finding13 takes three comparisons; rejecting17 also takes three. n congruent keys give O(n) worst case. Average O(1) needs hash/load assumptions.
2. A maintained balanced ordered tree gives O(log n) worst-case search and range queries. Whether it is preferable for exact-key lookup also depends on costs and synchronization.
3. Unlinking removes future list reachability but does not destroy acquired references. Immediate free/reuse permits use-after-free. Reclaim under a suitable lock, reference-count protocol, or reader-compatible grace period. Stale values are different from invalid memory.

Query cost and lifetime are independent obligations.

OS01-E04 Overlapping two jobs

1. A uses CPU 0–2, I/O2–8, CPU 8–10; B runs10–14. Total14, CPU utilization8/14=4/7.
2. A0–2, B2–6, idle6–8, A8–10; I/O remains2–8. Total10, utilization8/10=0.8. The saved4 units are B's overlap with I/O.
3. No ready B, shared-I/O queueing, switching/synchronization overhead, or memory pressure changes the premises. One CPU still performs8 total CPU units without parallel CPU bursts.

List idle intervals explicitly.

OS01-E05 Cache and NUMA denominators

1. Additional-penalty model gives1+0.1×50=6 cycles. Total-miss-cost model gives0.9×1+0.1×50=5.9 cycles. These formulas use different contracts.
2. 0.8×100+0.2×180=116 ns;0.2×100+0.8×180=164 ns.
3. No. Whole-program time also depends on memory stalls' baseline-time fraction, overlap, caches, bandwidth, and all other work. One subsystem's cost does not automatically equal total execution time.

Keep units and premises.

Introduction solutions (continued)

OS01-E06 Equal VA, unequal data

1. VPN=1, offset0x234=564. P gives0x7000+0x234=0x7234; Q gives0x9234.
2. P's permission allows the write. Q takes a permission fault. Do not allocate a new page merely to bypass prohibited writing. COW requires separate policy markers and conditions absent here.
3. No. A remaining mapping can otherwise access a frame reused for someone else's data. Old references, TLB entries, and I/O must no longer use the frame before it enters the free pool.

Residency, permission, and sharing are distinct.

OS01-E07 Timer and fairness premises

1. With fixed rotation, no higher-priority insertion, and terminating service, a just-missed job waits at most(n−1)q+n c. Changing n or unbounded nonpreemptible execution invalidates the bound.
2. Each q of work incurs c switching. The fraction is0.1/4.1=1/41≈2.44%. It is not a measurement of all context switches.
3. No. A nonterminating handler prevents return to a progressing scheduler. Interrupt serviceability, bounded kernel blocking, and a selection policy without permanent starvation are also needed.

Hardware control recovery does not replace scheduler policy.

OS01-E08 Lost update and BST range

1. T1load0, T2load0, T1store1, T2store1 produces1. A single CPU can switch between steps. Suitable atomic RMW or mutex changes the update to an indivisible specification.
2. Although12>parent5, it belongs to root 10's left subtree while12>10. Search follows right20 then its empty left child, missing existing12. The invariant covers whole subtree ranges.
3. One bit per block gives2^20 bits/8=2^17 B=128 KiB. This is logical bitmap size, not one bit of physical I/O per operation.

Local comparisons do not establish a general invariant.

Introduction solutions (continued)

OS01-E09 Deadlines and platform boundaries

1. A0–1, B1–3, idle3–4, A4–5, idle5–6, B6–8, A8–9, idle9–12. A finishes1,5,9 before4,8,12. B finishes3,8 before6,12. LCM12 and no unfinished boundary work permit repetition.
2. No. U=1/4+2/6=7/12 states demand only. An arbitrary unfair scheduler can idle or choose wrongly. Feasibility, algorithmic guarantees, and actual premises need separate proofs.
3. VMs normally have guest kernels and a VMM boundary. Conventional containers share a host kernel and isolate views through namespaces/resource controls. They do not each necessarily have an independent OS kernel.

Timing correctness is stronger than a low mean.

OS01-E10 API, version, and source availability

1. No. The OSI definition also requires specified redistribution and derived-work rights. Source availability alone is insufficient; inspect each component's license.
2. No. Fix version, architecture, and configuration; consult allowed API outcomes and implementation, then compare general invariants and condition-specific observations. One success excludes neither legal error branches nor races/failures.
3. Success may reach only volatile cache. Define file, metadata, device-cache flush ordering, compliant device behavior, necessary namespace persistence, and the failure model. A successful return alone is insufficient.

Evidence categories cannot replace each other.

Introduction solutions (continued)

Verification code appendix: intro_models.py

```
"""Standalone OS1 model: exact arithmetic and explicit sequential events."""
from fractions import Fraction
from itertools import permutations


def average_cache(hit=Fraction(9,10),lookup=1,extra=50):
    if not 0<=hit<=1 or lookup<0 or extra<0:raise ValueError('invalid cost')
    return lookup+(1-hit)*extra


def numa(local_fraction):
    if not 0<=local_fraction<=1:raise ValueError('ratio')
    return local_fraction*100+(1-local_fraction)*180


def translate(va,pages,page_size=4096,write=False):
    if va<0 or page_size<=0:raise ValueError('address')
    vpn,offset=divmod(va,page_size)
    if vpn not in pages:raise KeyError('not present')
    frame,writable=pages[vpn]
    if write and not writable:raise PermissionError('write fault')
    return frame*page_size+offset


def two_jobs():
    serial=[('A',0,2),('idle',2,8),('A',8,10),('B',10,14)]
    overlap=[('A',0,2),('B',2,6),('idle',6,8),('A',8,10)]
    util=lambda rows:Fraction(sum(end-start for who,start,end in rows if
who!='idle'),rows[-1][2])
    return
{'serial':serial,'overlap':overlap,'serial_utilization':util(serial),'overlap_utilization':ut
il(overlap),'io':('A',2,8)}


def lost_updates():
    """All orderings preserving each thread's load/add/store order.
    +1 is folded into store of the saved load; no memory-model claim.
    """
    events=['L1','S1','L2','S2'];outcomes={}
    for order in permutations(events):
        if order.index('L1')>order.index('S1') or
order.index('L2')>order.index('S2'):continue
        memory=0;registers={}
        for event in order:
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Introduction solutions (continued)

```
            op,thread=event
            if op=='L':registers[thread]=memory
            else:memory=registers[thread]+1
        outcomes[order]=memory
    return outcomes


def bst_valid(node,lo=None,hi=None):
    if node is None:return True
    key,left,right=node
    if (lo is not None and key<=lo) or (hi is not None and key>=hi):return False
    return bst_valid(left,lo,key) and bst_valid(right,key,hi)


def edf_periodic(tasks,horizon):
    """Unit-tick preemptive model: (C, period, relative deadline), releases at0.
    No overhead/blocking; jobs may coexist. Tie by release then task id.
    """
    if horizon<0 or any(c<=0 or t<=0 or d<=0 for c,t,d in tasks):raise ValueError('task')
    jobs=[];timeline=[];completions=[]
    for now in range(horizon):
        for task,(cost,period,deadline) in enumerate(tasks):
            if
now%period==0:jobs.append({'task':task,'release':now,'deadline':now+deadline,'remaining':cost
})
        candidates=[job for job in jobs if job['remaining']>0]
        if not candidates:timeline.append(None);continue
        job=min(candidates,key=lambda j:(j['deadline'],j['release'],j['task']))
        job['remaining']-=1;timeline.append(job['task'])
        if
job['remaining']==0:completions.append((job['task'],job['release'],now+1,job['deadline']))
    return timeline,completions,jobs


if __name__=='__main__':
    print('cache cycles',average_cache())
    print('NUMA ns',numa(Fraction(4,5)),numa(Fraction(1,5)))
    print(two_jobs())
    print('PA',hex(translate(0x1234,{1:(7,True)})))
    print('counter outcomes',set(lost_updates().values()))
    print('EDF',edf_periodic([(1,4,4),(2,6,6)],12)[:2])
```

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study